Maßnahmen am Simulationsmodell SimRepair und am PSP

Probabilistic Suffix Prescription, SCOPE_WORKSPACE · Stand: 20.10.2026

Dieses Dokument fasst alle Änderungen zusammen, die am Simulationsmodell **SimRepair** und an der Einbindung des **Probabilistic Suffix Predictors (PSP, U-ED-LSTM)** als kausaler Schätzer in SCOPE vorgenommen wurden. Grundlage ist der Git-Stand des Repositories: Commit *93cf1dd* (04.10., Ausgangsstand), Commit *11f1c35* (09.10., PSP-Modell und SimRepair-Pipeline) sowie die danach noch nicht committeten Änderungen, einschließlich der Fehlerbehebungen vom 19. und 20.10. Nicht enthalten ist das separate Projekt *process-simulation-engine* (BPIC17).

Bereich	Kernpunkte
1 SimRepair	Gewinn als Outcome, deterministische Werkstatt-Policy, neue Kundenentscheidung mit Deadline, zwei Interventionen (Prozedur, Qualitätskontrolle), Datengenerierung mit Werkstatt- und RCT-Daten
2 PSP in SCOPE	Eigenes Netz und eigene Model Functions (S-Learner), Suffix-Tensoren, Training mit Teacher Forcing, Q-Werte aus 200 gezogenen Suffixen je Fall und Aktion
3 PSP-Repo	Optionale Erweiterungen (Decoder-Kontext, Verlustvarianten, bestes Modell speichern); in SCOPE nicht genutzt
4 Fehlerbehebungen	Falsche Kontroll-Entscheidungspunkte bei der QC-Stage, Tuning pro Stage für DTR
5 Ergebnisse	Nach dem Fix: PSP und XGBoost gleichauf (etwa +450 bis +540), beide klar über der besten festen Policy
6 Offene Punkte	Stage-0-Tuning PSP (DTR) läuft, kleines Testset, ein Seed, kleinere Code-Probleme

1 Simulationsmodell SimRepair

SimRepair simuliert einen Fahrradreparatur-Prozess als Petri-Netz (pm4py) mit diskreter Ereignissimulation (simpy). Ein Fall durchläuft: Fall anlegen → Prozedur wählen (Standard oder Priority) → Mitarbeiter und Material → Reparatur (ggf. Nachreparatur) → Qualitätskontrolle oder Versand → Kundenannahme oder -ablehnung. Das Prozessmodell liegt zusätzlich als BPMN vor (*simrepair_process.bpmn/.png*, Skript *render_bpmn_png.py*).

1.1 Outcome-Funktion

Alle Größen in Tagen, in derselben Einheit wie die Zeitkosten. Datei: *SimRepair/activity_execution.py*.

	Vorher (93cf1dd)	Jetzt
Annahme	Outcome 0	Erlös (10 + 2·Fahrradwert + Schwere) × (1 − 0,3·Unsicherheit) × Qualitätsfaktor − verbrauchte Tage
Ablehnung	Outcome −100, Dauer 100 Tage	− verbrauchte Tage, Dauer 0
Priority	–	fester Aufschlag von 5 Tagen auf die Kosten
Qualitätsfaktor	–	1 + 0,5 · (produzierte − geforderte Qualität), begrenzt auf ±0,4

Geforderte Qualität = 0,6·Schwere + 0,4·Fahrradwert, produzierte Qualität = 0,6·Mitarbeiterkompetenz + 0,4·Materialqualität (jeweils auf [0, 1] normiert, Hilfsfunktionen in *workshop_policy.py*). Der Erlös aus Commit 11f1c35 stammt aus der Gewinnlogik von SimBank; Priority-Aufschlag und Qualitätsfaktor kamen danach hinzu.

1.2 Werkstatt-Policy (Verhaltenspolitik)

Die stochastischen Sigmoid-Regeln wurden durch einfache, deterministische Werkstattregeln ersetzt, analog zu den Schwellen-Policies in SimBank. Datei: *SimRepair/workshop_policy.py*.

Entscheidung	Vorher	Jetzt
Verfügbaren Mitarbeiter nehmen	Sigmoid aus Kompetenz, Schwere, Wert	nehmen, wenn Kompetenz ≥ geforderte Qualität − 0,1

Entscheidung	Vorher	Jetzt
Mitarbeiter / Material wählen	Sigmoid aus Schwere, Wert, Unsicherheit	passend zur geforderten Qualität
Qualitätskontrolle	Sigmoid aus Unsicherheit, Schwere, Wert, Zeit	QC genau dann, wenn Unsicherheit > 0,5
Prozedur (Standard/Priority)	Sigmoid aus Wert, Schwere, Unsicherheit	unverändert

1.3 Umwelt und Kundenentscheidung

Datei: *SimRepair/environment.py*.

- **Fehlerhafte Reparatur:** sigmoid(−2 + 12·(gefordert − produziert)) statt sigmoid(−1 + 5·(…)). Selten (12 %), wenn Mitarbeiter und Material passen, fast sicher, wenn sie deutlich zu schwach sind.
- **Material vergessen:** sigmoid(−2 + 6·(Schwere − Kompetenz)) statt sigmoid(−1 + 2,5·Schwere − 3·Kompetenz).
- **Kundenannahme** nach dem Muster von SimBank: Deadline = 10 + 2·Geduld Tage, danach praktisch sichere Ablehnung. Sonst p = 0,5 + 0,25·Zeitfaktor + 0,2·Qualitätsfaktor + 0,05·Freundlichkeitsfaktor; ungünstige Fälle (p < 0,5) werden auf [0; 0,15), günstige auf [0,85; 1] abgebildet. Vorher: eine einzelne Sigmoid-Funktion.

1.4 Interventionen und Datengenerierung

Dateien: *SimRepair/simRepair_run.py* (neu in 11f1c35), *simulation.py*, *generate_sequential_simbank.py*.

Stage	Intervention	Aktionen (Kontrolle / Treatment)	Entscheidungspunkt
0	choose_procedure	start_standard / start_priority	nach initiate_case
1	conduct_qc	shipping / conduct_qc	nach repair_* bzw. improve_rework; QC wird genau einmal entschieden (action_depth 4 → 1)
–	choose_employee	Kompetenz 1–10	deaktiviert (auskommentiert)

- **Trainingsdaten:** Ein Werkstatt-Datensatz (Policy aus 1.2) und ein RCT-Datensatz (zufällige Aktionen) werden mit *set_delta* gemischt; δ ist der Anteil der Werkstattfälle. Nur fallweises Confounding (*confounding_type='case'*). Es wird neu gemischt, bis jede Aktion in mindestens zwei Fällen vorkommt.
- **Merkmale:** customer_patience ist neu ein Fallattribut und wird skaliert (Fallattribute: bike_value, repair_severity, customer_patience).
- **Evaluation:** Für jede feste Aktionskombination, die Werkstatt-Policy (bank), eine zufällige Policy und die Optimal-Policy werden Testfälle mit gleichen Seeds simuliert. Jede Stage wird nur am ersten Entscheidungspunkt entschieden, an weiteren Punkten derselben Stage gilt die Kontrollaktion.
- **Simulator:** Sind außerhalb des Interventionszeitpunkts alle Alternativen Interventionsaktivitäten (z. B. shipping/conduct_qc), wird die Kontrollaktion genommen.
- **Ablage:** Eval-Daten liegen getrennt nach Trainingsgröße und δ (*data/SimRepair/<n>/<δ>/eval*).

2 PSP als kausaler Schätzer in SCOPE

Das PSP ersetzt in SCOPE das Outcome-Modell (Q-Funktion). Es sagt zu einem Prefix und einer Aktion den restlichen Prozessverlauf (Suffix) samt Outcome probabilistisch voraus. Hauptdatei: *src/utils/model_tools/psp_model_functions.py*.

2.1 Einbindung und Auswahl

- **Auswahl:** *--model_categories ml dl dl ml --model_specifics xgb psp xgb* (zweiter Eintrag = Outcome-Modell). Das PSP verwendet automatisch die Tensor-Kodierung.
- **Methoden:** läuft als S-Learner unter *separate-S-reg-none* und unter DTR (*dtr-S-reg-R*, Backward Induction über SCOPEFunctions). Das Outcome-Ziel ist das von SCOPE angepasste Y.

- **Model Functions:** *PSPCausalRegressor* liefert die von SCOPE erwartete Schnittstelle (*.model*, *get_loss*, *forward*); *get_model_functions* wählt ihn statt *DLCausalRegressor*.
- **Import:** Encoder und Decoder kommen unverändert aus *probabilistic_suffix_prediction_U-ED-LSTM* (MC-Dropout-LSTM-Zellen). Weil beide Repos ihr Paket *src* nennen, wird nur *src/model* in den Suchpfad gelegt.

2.2 Netzarchitektur (Klasse PSP)

- **Encoder:** Aktivität und weitere kategoriale Event-Attribute (z. B. process_type) als Embeddings; numerische Attribute als Wert plus Fehlt-Kennzeichen (Platzhalter −100); Treatment am letzten Prefix-Event (S-Learner); Fallattribute als statischer Input.
- **Decoder** im Merkmalsraum des Suffixes; Start mit dem letzten Prefix-Event (SOS). Köpfe: Aktivität inkl. EOS, weitere kategoriale Attribute, Fehlt-Kennzeichen je numerischem Attribut, numerische Attribute (Mittelwert und Log-Varianz), Outcome (Mittelwert und Log-Varianz).
- **Training:** Teacher Forcing über das Suffix, optional Scheduled Sampling; das Outcome wird nur am EOS-Event gelernt.
- **Rollout:** zieht Suffixe autoregressiv bis EOS; jede Ziehung hat eigene, über alle Schritte feste Dropout-Masken. Wachsende Attribute (elapsed_time) fallen nie; ohne EOS wird nach max_process_len + 1 Schritten abgeschnitten.

2.3 Verlust, Training und Q-Werte

- **Verlust:** Cross-Entropy für Aktivität (inkl. EOS) und weitere kategoriale Attribute, binäre Cross-Entropy für Fehlt-Kennzeichen, Loss Attenuation (Kendall & Gal) für numerische Attribute und Outcome, L2-Regularisierer 10^{-4}; optional GradNorm-Gewichtung je Aufgabe.
- **Trainingsschleife** (*model_training.py*), nur für das PSP: Learning-Rate-Scheduler (Faktor 0,3, Geduld 8, min. 10^{-5}), Scheduled Sampling, Tensorboard, Losses je Attribut, periodisches Speichern.
- **Q-Werte:** Für jede Aktion werden je Fall 200 Suffixe gezogen (*psp_mc_samples*); der Q-Wert ist der Mittelwert der Outcomes am EOS-Event. Zusätzlich werden Streuung der Mittelwerte und aleatorische Varianz zurückgegeben.
- **Suffix-Auswertung** nach jedem Training (*psp_evaluation.py*, neu): wie im PSP-Repo Mittelwert- und Wahrscheinlichkeitsvorhersage, Damerau-Levenshtein-Ähnlichkeit, Suffixlänge, letzter Wert wachsender Attribute, PIT, Intervalle und Outcome-Kennzahlen.
- **Konfiguration:** alle Parameter mit Präfix *psp_* in *config/config.py*.

2.4 Datenaufbereitung für das PSP

- **Suffix-Tensoren** (*tensor_prep.py*): zu jedem Prefix die restlichen Events des Falls (*X_suffix*), mit EOS-Event und Outcome nur am EOS, sowie *suffix_len*. Datensatz und Dataloader reichen beides durch (*data_load_tools.py*).
- **Weitere Spalteninformation** für das Netz: *event_cols_encoded*, *event_cat_cols*, *suffix_cols_encoded*.
- **Treatment-Spalte** wird jetzt innerhalb eines Falls verschoben (vorher über Fallgrenzen hinweg); zusätzlich werden Interventionen auf Attributspalten unterstützt.

3 Änderungen im PSP-Repository

Nicht committete Änderungen in *probabilistic_suffix_prediction_U-ED-LSTM*. Alle sind optional und standardmäßig aus. Das PSP in SCOPE baut Encoder und Decoder direkt auf und nutzt diese Erweiterungen derzeit nicht.

- **Event-Log-Loader:** akzeptiert einen DataFrame direkt statt nur einer CSV-Datei.
- **Decoder-Kontext:** Optional gehen der letzte Encoder-Zustand und/oder die eingebetteten statischen Attribute in *jeden* Decoder-Schritt ein (*decoder_context*, *decoder_static*), damit die Prefix-Information über lange Suffixe erhalten bleibt; wird im Checkpoint mitgespeichert.
- **Trainer:** Wahl des kategorialen Verlusts (Loss Attenuation oder normale Cross-Entropy, optional mit Klassengewichten), Teacher-Forcing-Abbau erst ab einem Anteil der Epochen, Speichern des Modells mit

dem kleinsten Validierungsverlust.

4 Fehlerbehebungen vom 19. und 20.10.2026

4.1 Falsche Kontroll-Entscheidungspunkte bei der QC-Stage

Problem: Für *conduct_qc* sind repair_priority, repair_standard und improve_rework als Kontroll-Aktivitäten eingetragen. Das Preprocessing legte an *jedem* dieser Events ohne vorheriges Treatment eine Kontrollzeile mit T = 0 (Shipping) an, auch bei Reparaturen, auf die eine Nachreparatur folgt statt einer QC-Entscheidung.

- Von 4769 Stage-1-Trainingszeilen waren etwa 770 solche Scheinzeilen, 296 davon aus Fällen mit tatsächlicher QC.
- Weil die Werkstatt bei Unsicherheit > 0,5 immer QC macht, landeten nachreparierte Fälle mit schlechtem Outcome bei hoher Unsicherheit als „Shipping“ in den Daten. Shipping sah dort künstlich schlecht aus, die Modelle lernten „QC bei hoher Unsicherheit“, obwohl QC in SimRepair praktisch immer schadet (wahrer Effekt −3 bis −7 je Fall).
- Betroffen waren beide Kodierungen: *agg_prep.py* (XGBoost) und *tensor_prep.py* (PSP).

Behebung: Eine Kontrollzeile entsteht nur noch, wenn direkt danach eine der Aktionen der Intervention folgt. Bei Interventionen mit *do_nothing* (SimBank), deren Kontrollaktion nicht im Log steht, bleibt das Verhalten unverändert. Prüfung: jetzt genau eine Zeile pro Fall und Stage, T stimmt zu 100 % mit der echten Entscheidung überein; Stage-0- und Stage-1-Zeilen sind über die Fallnummer ausgerichtet.

4.2 Tuning pro Stage für DTR

- Neues Argument *--already_tuned_stages* (*mini_tools.py*, *method_main.py*): Für die angegebenen Stages werden gespeicherte Tuning-Ergebnisse geladen, die anderen Stages werden getunt.
- Wird bei DTR die letzte Stage geladen, setzt SCOPE jetzt trotzdem die Zielwerte für die Backward Induction (echtes Y). Für mittlere Stages bricht der Code bewusst mit einer Fehlermeldung ab.
- Für den DTR-Lauf des PSP wurden die Tuning-Parameter aus *separate* übernommen; Methodenfelder wurden angepasst und die veralteten *prev_*_model_params* entfernt.

5 Ergebnisse

SimRepair, 5000 Trainingsfälle, δ = 0,5, 2 Stages, 500 Testfälle, Seed 42. Profit = Summe der Outcomes über alle Testfälle.

Methode / Modell	Vor dem Fix	Nach dem Fix	Priority (Stage 0)	QC (Stage 1)
separate · PSP	−399	+446	80 %	29 % → 5 %
separate · XGBoost	−425	+477	72 %	38 % → 8 %
dtr-S-reg-R · PSP *	–	+536	27 %	5 %
dtr-S-reg-R · XGBoost	–	+529	66 %	8 %

Vergleichs-Policy	Profit
beste feste Policy (Priority, dann Shipping)	−326
Werkstatt-Policy (bank)	−1742
zufällige Policy	−2046
realistische Obergrenze (Modell auf echten kontrafaktischen Outcomes, Kreuzvalidierung)	≈ +530
optimal (bester simulierter Verlauf je Fall)	+2811

* Stage 0 des PSP unter DTR nutzt noch die Hyperparameter aus einem kurzen, älteren Tuning (siehe 6).

- Nach dem Fix schlagen beide Modelle die beste feste Policy deutlich (gepaarter Bootstrap, 95 %-Intervall für PSP − fest: +319 bis +1250).
- PSP und XGBoost liegen gleichauf; alle Unterschiede zwischen den vier Varianten liegen im Rauschen (z. B. PSP − XGBoost unter DTR: +7, Intervall −394 bis +410).

- Der Optimal-Wert ist kein realistischer Maßstab: Er nutzt den Zufall der Kundenannahme aus (angenommen ≈ +7, abgelehnt ≈ −20 je Fall).
- Unter DTR trifft XGBoost die Stage-0-Entscheidung plausibel (Priority bei leichten, Standard bei schweren Reparaturen). Das PSP wählt fast überall Standard und reagiert kaum auf die Schwere der Reparatur.

6 Offene Punkte

- **Stage-0-Tuning des PSP unter DTR** (42 Versuche) läuft seit 20.10., 08:35, in tmux (*psp_tune0*); Log: *.tmp/psp_5000_50_dtr_tune0.log*. Die vorherigen DTR-Ergebnisse sind in *.tmp/dtr_stage0_oldtuning/* gesichert.
- **Belastbarkeit:** nur ein Seed und 500 Testfälle (Standardfehler einer Policy-Summe ≈ ±300). Nötig sind mehrere Iterationen und ein größeres Testset.
- **Dateinamen:** Bei ML-Modellen fehlt der Modellname in den Eval-Dateien (*…_outcome_ml_0_…*); ein weiterer ML-Lauf würde XGBoost-Ergebnisse überschreiben.
- **make_dirs:** Zwei gleichzeitig startende Läufe können beim Anlegen desselben Ordners abstürzen.
- **Debug-Ausgabe:** *tensor_prep.py* schreibt bei jedem Aufruf die gesamten Daten nach *src/readable.txt* (fester absoluter Pfad).
- **PSP.forward** liest das Outcome nach einem Decoder-Schritt ab und passt nicht zum Training; die Q-Werte nutzen deshalb den Rollout (*predict*).